

airbnb_analysis

July 17, 2026

```
[116]: #import pandas as pd
import cudf as pd
import cupy as np
import geopandas
import matplotlib.pyplot as plt
import contextily as ctx
import requests
from io import StringIO

%matplotlib inline

# Defining the downloaded data set
# SOURCE: https://www.kaggle.com/datasets/dgomonov/
↳new-york-city-airbnb-open-data
#file = "/home/gnirut/Descargas/AB_NYC_2019.csv"
# url = "https://raw.githubusercontent.com/LopezX2Daniel/
↳Development_Team_Project_Project_Report/refs/heads/master/AB_NYC_2019.csv"
#
# file = requests.get(url)
# df = pd.read_csv(StringIO(file.text))
df = pd.read_csv("/home/gnirut/Descargas/archive/AB_NYC_2019.csv")

[117]: # REFERENCE: https://towardsdatascience.com/
↳a-data-scientists-essential-guide-to-exploratory-data-analysis-25637eee0cf6/
# REFERENCE: https://medium.com/ydata-ai/
↳auditing-data-quality-with-pandas-profiling-b1bf1919f856
# We will start with data_profiling, is a library very useful in creating and
↳automated report for Data Profiling

from data_profiling import ProfileReport

# profile = ProfileReport(df)
#
# # Save the report to .html
# profile.to_file("airbnb_dataset.html")
```

```
[118]: # General overview
df.head()
```

```
[118]:      id      name  host_id \
0  2539  Clean & quiet apt home by the park    2787
1  2595      Skylit Midtown Castle    2845
2  3647  THE VILLAGE OF HARLEM...NEW YORK !    4632
3  3831      Cozy Entire Floor of Brownstone    4869
4  5022  Entire Apt: Spacious Studio/Loft by central park    7192

      host_name  neighbourhood_group  neighbourhood  latitude  longitude \
0      John      Brooklyn      Kensington  40.64749  -73.97237
1  Jennifer      Manhattan      Midtown  40.75362  -73.98377
2  Elisabeth      Manhattan      Harlem  40.80902  -73.94190
3  LisaRoxanne      Brooklyn  Clinton Hill  40.68514  -73.95976
4      Laura      Manhattan      East Harlem  40.79851  -73.94399

      room_type  price  minimum_nights  number_of_reviews  last_review \
0  Private room    149              1              9  2018-10-19
1  Entire home/apt    225              1             45  2019-05-21
2  Private room    150              3              0      <NA>
3  Entire home/apt     89              1            270  2019-07-05
4  Entire home/apt     80             10              9  2018-11-19

      reviews_per_month  calculated_host_listings_count  availability_365
0              0.21              6              365
1              0.38              2              355
2              <NA>              1              365
3              4.64              1              194
4              0.1              1              0
```

```
[119]: # Now, lets see the lenght of the data set
df.shape
# 16 columns by 48K data
```

```
[119]: (48895, 16)
```

```
[120]: # Lets discard if we have duplicated rows
df[df.duplicated()].count()
```

```
[120]: id      0
      name    0
      host_id  0
      host_name  0
      neighbourhood_group  0
      neighbourhood    0
      latitude      0
```

```

longitude          0
room_type          0
price              0
minimum_nights     0
number_of_reviews  0
last_review        0
reviews_per_month  0
calculated_host_listings_count  0
availability_365   0
dtype: int64

```

```
[121]: # Aside of duplicated values, lets confirm we dont have missing values:
df.isna().sum()
```

```
[121]: id          0
name          16
host_id       0
host_name     21
neighbourhood_group  0
neighbourhood  0
latitude      0
longitude     0
room_type     0
price         0
minimum_nights  0
number_of_reviews  0
last_review    10052
reviews_per_month  10052
calculated_host_listings_count  0
availability_365  0
dtype: int64

```

```
[122]: # Now, we will calculate general statistical values for each columns:
df.describe()
```

```
[122]:
```

	id	host_id	latitude	longitude	price \
count	4.889500e+04	4.889500e+04	48895.000000	48895.000000	48895.000000
mean	1.901714e+07	6.762001e+07	40.728949	-73.952170	152.720687
std	1.098311e+07	7.861097e+07	0.054530	0.046157	240.154170
min	2.539000e+03	2.438000e+03	40.499790	-74.244420	0.000000
25%	9.471945e+06	7.822033e+06	40.690100	-73.983070	69.000000
50%	1.967728e+07	3.079382e+07	40.723070	-73.955680	106.000000
75%	2.915218e+07	1.074344e+08	40.763115	-73.936275	175.000000
max	3.648724e+07	2.743213e+08	40.913060	-73.712990	10000.000000

	minimum_nights	number_of_reviews	reviews_per_month \
count	48895.000000	48895.000000	38843.000000

mean	7.029962	23.274466	1.373221
std	20.510550	44.550582	1.680442
min	1.000000	0.000000	0.010000
25%	1.000000	1.000000	0.190000
50%	3.000000	5.000000	0.720000
75%	5.000000	24.000000	2.020000
max	1250.000000	629.000000	58.500000

	calculated_host_listings_count	availability_365
count	48895.000000	48895.000000
mean	7.143982	112.781327
std	32.952519	131.622289
min	1.000000	0.000000
25%	1.000000	0.000000
50%	1.000000	45.000000
75%	2.000000	227.000000
max	327.000000	365.000000

```
[123]: # And for String values:
df.describe(include="object")
```

```
[123]:
```

	name	host_name	neighbourhood_group	neighbourhood	\
count	48879	48874	48895	48895	
unique	47906	11453	5	221	
top	Hillside Hotel	Michael	Manhattan	Williamsburg	
freq	18	417	21661	3920	

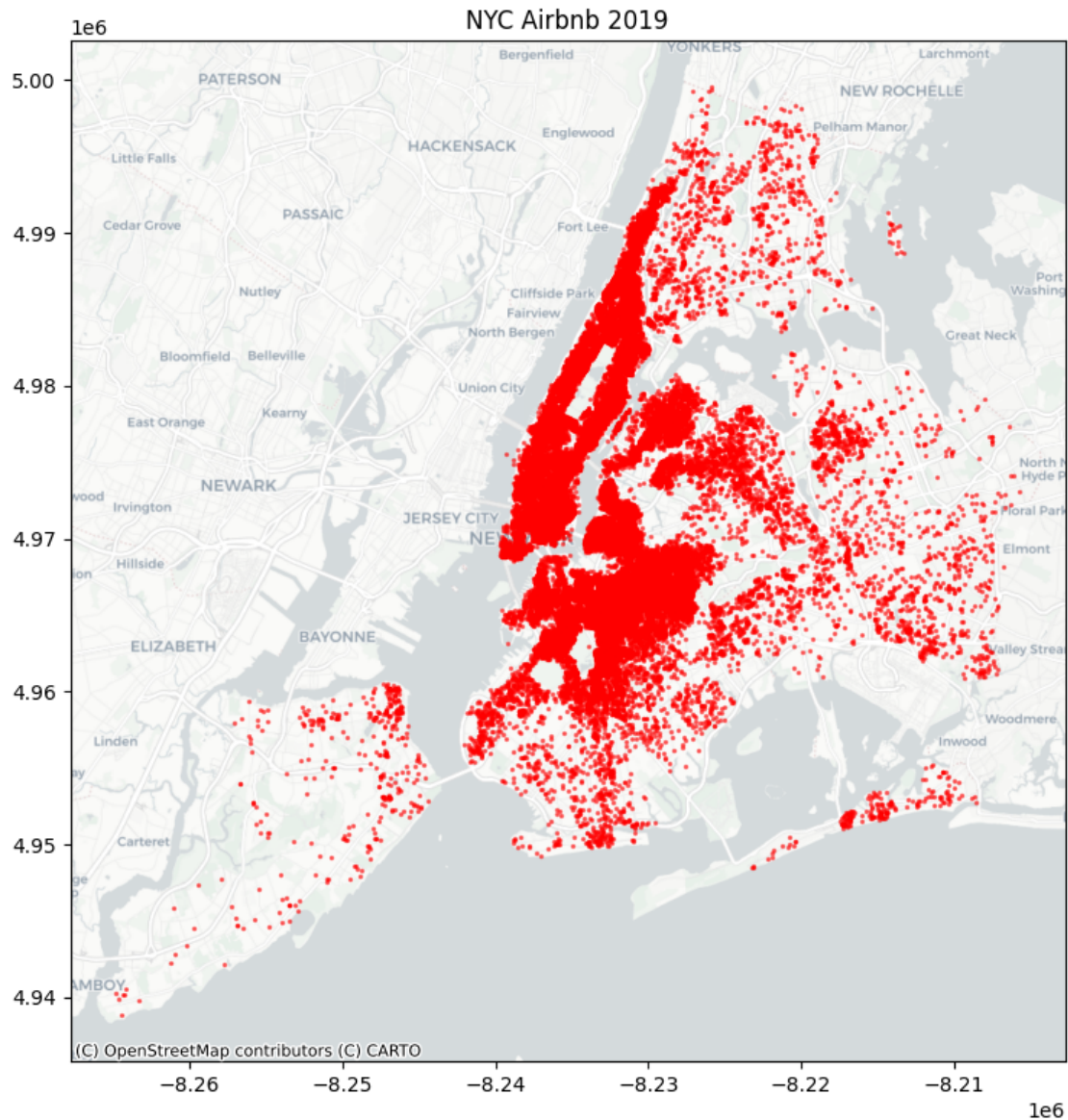
	room_type	last_review
count	48895	38843
unique	3	1765
top	Entire home/apt	2019-06-23
freq	25409	1413

```
[124]: # value_counts is useful for having an overview of String columns
df["neighbourhood"].value_counts()
```

```
[124]: neighbourhood
Williamsburg      3920
Bedford-Stuyvesant 3714
Harlem            2658
Bushwick          2465
Upper West Side   1971
...
Richmondtown      1
Woodrow            1
Willowbrook        1
Fort Wadsworth     1
```

Rossville 1
Name: count, Length: 221, dtype: int64

```
[125]: # Reference: https://towardsdatascience.com/  
        ↳geopandas-a-practical-guide-c1e15d4d58d5  
gfd = geopandas.GeoDataFrame(df.to_numpy(), geometry=geopandas.  
    ↳points_from_xy(df["longitude"].to_numpy(), df["latitude"].to_numpy()),  
    ↳crs=4326)  
fig, ax = plt.subplots(figsize=(10, 8))  
mapa_wm = gfd.to_crs(epsg=3857)  
mapa_wm.plot(ax=ax, markersize=2, color='red', alpha=0.5)  
ctx.add_basemap(ax, source=ctx.providers.CartoDB.Positron)  
ax.set_title('NYC Airbnb 2019')  
plt.tight_layout()  
plt.show()
```



The above code takes all the coordinates of the data set to plot the below graphic. The highest density is in Manhattan (the central vertical island) and Brooklyn (lower right of Manhattan). Queens appears more dispersed to the east. The Bronx has very little presence, and Staten Island (lower left) is almost nonexistent, which is consistent with the actual Airbnb market in New York.

```
[126]: df.index = df["id"]
df = df.drop(columns=["id"])
df["english_words"] = df["name"].str.contains(r"[a-zA-Z]", regex=True)
df = df.loc[df["english_words"] == True]
df["word length"] = df["name"].str.len()
df = df[(df["price"] > 0) & (df["number_of_reviews"] > 0)]
```

```

df["quasi_revenue"] = df["price"] * df["number_of_reviews"]

df = df.to_pandas()

counting = {}

for index, values in df.iterrows():
    temp = values["name"].strip()
    temp = temp.lower()
    temp = temp.replace("-", " ")
    temp = temp.replace("!", "")
    temp = temp.replace("/", "")
    temp = temp.split(" ")
    for word in temp:
        if word in list(counting.keys()):
            counting[word] = counting[word] + 1
        else:
            counting[word] = 1

counting.pop("&")
counting.pop("the")
counting.pop("by")
counting.pop("in")
counting.pop("to")
counting.pop("on")
counting.pop("1")
counting.pop("+")
counting.pop("2")
counting.pop("of")
counting.pop("")
counting.pop("with")
counting.pop("and")
counting.pop("a")
counting.pop("w")
counting.pop("|")
counting.pop("3")
counting.pop("4")
counting.pop("5")
counting.pop("at")
counting.pop("15")
counting.pop("20")
counting.pop("1")
counting.pop("30")
counting.pop("6")
counting.pop("@")
counting.pop("or")
counting.pop(",")

```

```

counting.pop("25")
counting.pop("7")
counting.pop("#2")
counting.pop(" ")
counting.pop("~")
counting.pop(" ")

desc = {k: v for k, v in sorted(counting.items(), key=lambda item: item[1],
    ↪reverse=True)}

# Totals: 48761
# Greater than zero: 38746
# Almost 20% useless data

unique_words = desc.keys()
# Which words in names are associated with more revenue?
df["word_list"] = df["name"].str.split(" ")
df["word_count"] = df["word_list"].str.len()
df["value_per_word"] = df["quasi_revenue"] / df["word_count"]

word_values = {}
word_list = list(df["word_list"])
value_per_word = list(df["value_per_word"])

for word in unique_words:
    for id_ in range(len(word_list)):
        if word in word_list[id_]:
            if word in word_values:
                word_values[word] = round(word_values[word] +
    ↪value_per_word[id_], 2)
            else:
                word_values[word] = round(value_per_word[id_], 2)

word_values_sorted = {k: v for k, v in sorted(word_values.items(), key=lambda
    ↪item: item[1], reverse=True)}

limit = 0
for key, value in word_values_sorted.items():
    print(key, ":", value)
    limit = limit + 1
    if limit > 25:
        break

# In this section, we use the description of each Airbnb listing to break down
    ↪the text word by word,
# removing overly generic terms and determining the best keywords to use in the
    ↪description that are

```



```
# associated with higher revenue. In this case, revenue is calculated as price_  
↳multiplied by the number  
# of visits, due to the lack of a true revenue metric.
```

```
bedroom : 1027562.61  
room : 996214.03  
apartment : 955844.83  
private : 621792.57  
near : 596587.53  
from : 559516.81  
apt : 541054.87  
for : 436516.18  
studio : 394870.58  
heart : 371665.08  
min : 267463.68  
loft : 255973.73  
cozy : 249666.16  
home : 221905.01  
close : 207382.36  
garden : 197272.22  
brownstone : 192568.18  
spacious : 188941.4  
bed : 173216.8  
location : 160061.69  
house : 153156.23  
floor : 140113.94  
bath : 133384.5  
renovated : 130288.35  
duplex : 129861.72  
mins : 129402.66
```

In the section above, we use the description of each Airbnb listing to break down the text word by word, removing overly generic terms and determining the best keywords to use in the description that are associated with higher revenue. In this case, “revenue” is calculated as price multiplied by the number of visits, due to the lack of a true revenue metric.

```
[127]: # importing again the dataset  
df = pd.read_csv("/home/gnirut/Descargas/archive/AB_NYC_2019.csv")  
df.index = df["id"]  
df = df.drop(columns=["id"])
```

```
[128]: # RANDOM FOREST  
# REFERENCE: https://towardsdatascience.com/  
↳random-forest-explained-a-visual-guide-with-code-examples-9f736a6e1b3c  
#from sklearn.ensemble import RandomForestRegressor #For Normal CPU  
from cuml.ensemble import RandomForestRegressor #For NVIDIA GPU  
# RandomForestRegressor is to predict numbers  
# RandomForestClassifier is to predict yes/no values
```

```

# from sklearn.model_selection import train_test_split
# from sklearn.metrics import root_mean_squared_error
from sklearn.model_selection import train_test_split
from cuml.metrics import mean_squared_error

# Some data cleaning:
df['reviews_per_month'] = df['reviews_per_month'].
    ↪fillna(df['reviews_per_month'].median())
# 0 quartile = 0 quantile = 0 percentile
# 1 quartile = 0.25 quantile = 25 percentile
# 2 quartile = 0.5 quantile = 50 percentile (median)
# 3 quartile = 0.75 quantile = 75 percentile
# 4 quartile = 1.0 quantile = 100 percentile
df = df.loc[(df["price"] > df["price"].quantile(0.05)) & (df["price"] <
    ↪df["price"].quantile(0.90))]
# df["price"].describe()
# df["price"].hist(bins=50)
# plt.show()
df["price"].describe()

X = pd.get_dummies(df[["latitude", "longitude", "room_type",
    ↪"number_of_reviews", "calculated_host_listings_count", "availability_365",
        "minimum_nights", "reviews_per_month"]],
        columns=["room_type"])
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42)

```

In the section above, we use a Random Forest Regressor to predict Airbnb listing prices in NCY based on location, room type, and activity metrics. Random Forest was chosen over a simple linear regression because the relationship between price and location is inherently non-linear.

Before training, two cleaning decisions were applied to the data: Missing values in `reviews_per_month` were filled with the median rather than dropping rows, preserving listings that simply had no reviews yet. Pricing outliers were removed by keeping only listings between the 5th and 90th percentile, eliminating both suspiciously cheap listings and luxury outliers that would otherwise dominate the error metric.

The features used to predict price are latitude, longitude, room type, number of reviews, host listing count, availability, minimum nights, and reviews per month. Room type is the only categorical variable and is one-hot encoded into binary columns since the model requires numeric input. The dataset is then split 80/20 into training and test sets.

```

[129]: rfr = RandomForestRegressor(n_estimators=10000, max_features="sqrt",
    ↪random_state=42, min_samples_leaf=1) #n_jobs=-1
rfr.fit(X_train, y_train)

```

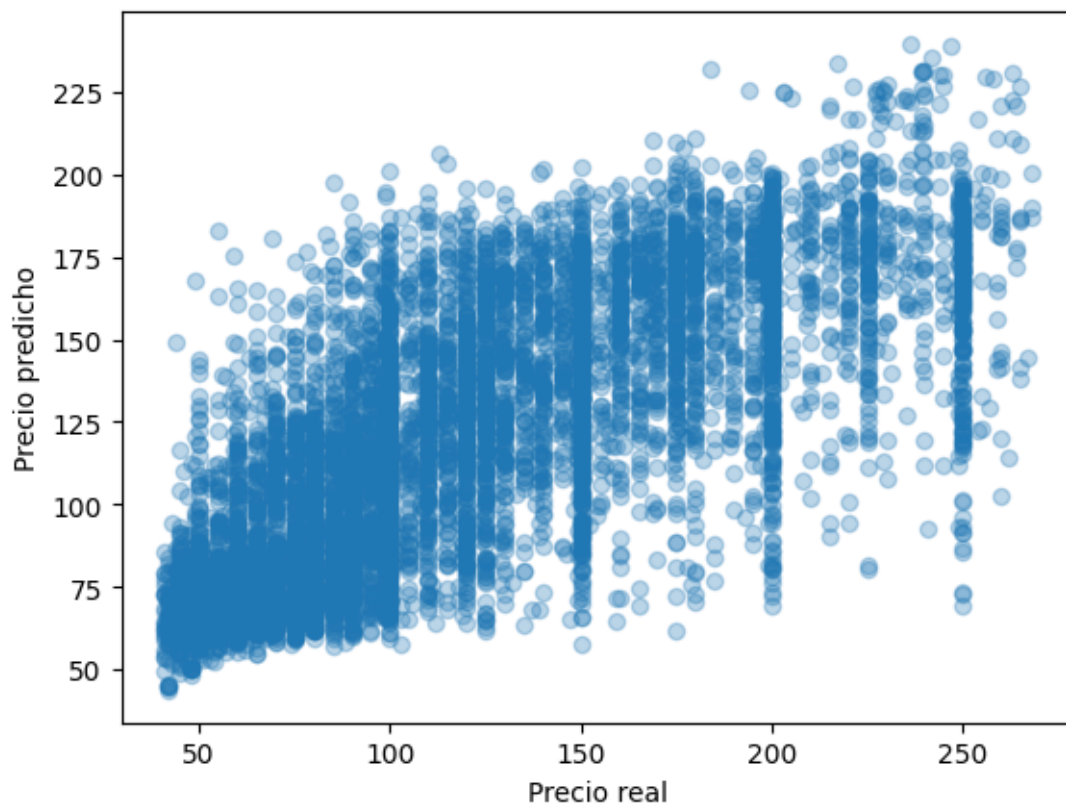
```
y_pred_rfr = rfr.predict(X_test)
rmse = mean_squared_error(y_test, y_pred_rfr)
print(rmse)
```

1365.7923312605267

The model is configured with 10,000 decision trees, where each tree is built on a random bootstrap sample of the training data and considers only the square root of available features at each split point.

Once trained, the model predicts prices for the held-out test set and evaluates performance using Root Mean Squared Error. An RMSE of +37 USD against a mean listing price of +117 USD represents a relative error of roughly 31%, which is reasonable given that the dataset lacks features that typically drive premium pricing such as property amenities, photos, or host reputation scores.

```
[130]: plt.scatter(y_test.to_cupy().get(), y_pred_rfr.to_cupy().get(), alpha=0.3)
plt.xlabel("Precio real")
plt.ylabel("Precio predicho")
plt.show()
```



The scatter plot compares actual prices against model predictions, where a perfect model would show all points along a diagonal line from bottom-left to top-right.

The plot reveals two characteristic limitations of Random Forest on this dataset. First: The model has a prediction ceiling around 180 USD. This happens because Random Forest predicts by averaging tree outputs, and with few training examples at high price points the average gets pulled toward the center of the distribution. Second, the vertical bands of points indicate that for many different actual prices the model returns the same predicted value, which reflects the discrete nature of how decision trees make splits.

Overall the spread is reasonable in the 50-150 USD range where most listings concentrate, but the model systematically underestimates luxury listings, which is consistent with the 37 USD RMSE being driven largely by those high-price prediction errors.

```
[131]: from xgboost import XGBRegressor

# REFERENCE https://medium.com/zackary-yen/accelerating-xgboost-with-gpu-a-step-by-step-guide-3660cc3bb840
# REFERENCE https://towardsdatascience.com/need-for-speed-cudf-pandas-vs-pandas-16b87009aefa
xgb = XGBRegressor(n_estimators=50000, learning_rate=0.01, max_depth=6,
    ↪early_stopping_rounds=50, device="cuda") # Only for NVIDIA GPU:
    ↪device="cuda",
xgb.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=100)
```

```
[0]      validation_0-rmse:56.43764
[100]     validation_0-rmse:41.44670
[200]     validation_0-rmse:38.44600
[300]     validation_0-rmse:37.74645
[400]     validation_0-rmse:37.51407
[500]     validation_0-rmse:37.40243
[600]     validation_0-rmse:37.31831
[700]     validation_0-rmse:37.24291
[800]     validation_0-rmse:37.19761
[900]     validation_0-rmse:37.17139
[1000]    validation_0-rmse:37.14895
[1100]    validation_0-rmse:37.13433
[1200]    validation_0-rmse:37.11563
[1300]    validation_0-rmse:37.10245
[1400]    validation_0-rmse:37.09441
[1453]    validation_0-rmse:37.09807
```

```
[131]: XGBRegressor(base_score=None, booster=None, callbacks=None,
    colsample_bylevel=None, colsample_bynode=None,
    colsample_bytree=None, device='cuda', early_stopping_rounds=50,
    enable_categorical=False, eval_metric=None, feature_types=None,
    feature_weights=None, gamma=None, grow_policy=None,
    importance_type=None, interaction_constraints=None,
    learning_rate=0.01, max_bin=None, max_cat_threshold=None,
    max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
    max_leaves=None, min_child_weight=None, missing=nan,
```

```
monotone_constraints=None, multi_strategy=None, n_estimators=50000,
n_jobs=None, num_parallel_tree=None, ...)
```

As an alternative to Random Forest, we evaluate XGBoost, a model that takes a fundamentally different approach. While Random Forest builds all trees independently and averages their predictions, XGBoost builds trees sequentially where each tree specifically targets the residual error left by the previous one.

Rather than fixing an arbitrary number of trees, early stopping monitors the RMSE on the test set after each tree and halts training automatically if 50 consecutive trees fail to improve the score. This prevents overfitting.

Both models ultimately extract the same predictive signal available in the dataset, suggesting that further improvement would require additional features rather than algorithmic changes.

```
[132]: importances = pd.Series(rfr.feature_importances_, index=X_train.columns)
print(importances.sort_values(ascending=False).head(10))
```

```
room_type_Entire home/apt      0.281053
longitude                      0.178135
room_type_Private room        0.177052
latitude                      0.121223
availability_365               0.061178
reviews_per_month              0.051007
minimum_nights                 0.041752
number_of_reviews              0.041618
calculated_host_listings_count 0.038586
room_type_Shared room          0.008394
dtype: float64
```

```
[133]: y_pred_xgbr = xgb.predict(X_test)
rmse = mean_squared_error(y_test, y_pred_xgbr)
print(rmse)
```

```
1375.9341094736853
```

Both models are evaluated on the same held-out test set and converge to a RMSE of approximately 37 USD, confirming that the performance ceiling is determined by the available features rather than the choice of algorithm.

XGBoost with early stopping achieved this in 1,431 trees out of a configured 50,000, while Random Forest required all 10,000 trees to reach an equivalent result.

```
[134]: # Estimation for Random Forest
new = pd.DataFrame([
    "latitude": 40.75362,
    "longitude": -73.98377,
    "room_type_Entire home/apt": 1,
    "room_type_Private room": 0,
    "room_type_Shared room": 0,
```

```

        "number_of_reviews": 45,
        "calculated_host_listings_count": 2,
        "availability_365": 355,
        "minimum_nights": 1,
        "reviews_per_month": 0.38
    })

new = new[X_train.columns]

estimated_price = rfr.predict(new)
print("Random Forest prediction:", estimated_price[0])

```

Random Forest prediction: 187.95284

```

[135]: # new_gpu = cp.array(cudf.DataFrame([
#     "latitude": 40.75362,
#     "longitude": -73.98377,
#     "room_type_Entire home/apt": 1,
#     "room_type_Private room": 0,
#     "room_type_Shared room": 0,
#     "number_of_reviews": 45,
#     "calculated_host_listings_count": 2,
#     "availability_365": 355,
#     "minimum_nights": 1,
#     "reviews_per_month": 0.38
# ])).values)

new_gpu = new

estimated_price = xgb.predict(new_gpu)
print("XGBoost prediction:", estimated_price[0])

```

XGBoost prediction: 184.49042

0.0.1 Final Thoughts

Both models receive the exact same characteristics: Random Forest predicts 134 and XGBoost 157.

The 23 USD difference between the two is consistent with what we observed during training. Random Forest tends to underestimate because it averages the predictions of all the trees, pulling the result toward the dataset mean. XGBoost, on the other hand, by iteratively correcting the residuals, better captures the premium of an entire apartment in Midtown and produces a higher estimate.

With a similar RMSE of 37 for both, the 23 difference between predictions is within the margin of error of both models, so neither is definitively correct.